

<https://github.com/gc-ghub/project-gc-industries-devops-superheros>

Please go through this GitHub repo. Each and every folder, commits and PR and GitHub Actions workflow files and pipeline runs. Readme file is not upto date. I will update it.

What do you think about the project and its owner.

I will build my project as independent one. Just wanted to check if kagent has done that already or how close it is to my project. Making my project independent would give me possibilities to add more capabilities in future and can help me build my thoughts clear.

That's exactly the right call. And your reasoning is sound on every count.

Why Independent Is Correct — Confirmed

Clarity of thought first. You said it yourself — building independently keeps your thinking clear. The moment you inherit someone else's framework, you start thinking within their constraints. Korion's architecture should emerge from what Korion needs to do, not from what kagent happens to support. That independence of thinking is what produces genuinely novel tools.

Future capability expansion without ceiling. Every major capability we've discussed — the topology canvas, ARIA's learning store, Krypt, the Platform Health Advisor, the canary decision intelligence, cost optimisation — none of these fit neatly inside kagent's model. If you'd built on kagent, you'd be constantly working around its boundaries. Building independently means the architecture grows with your vision, not against someone else's.

The CNCF case remains yours. An independent Korion has a clean, self-contained story. A kagent-dependent Korion is a plugin. The difference at CNCF submission is enormous.

kagent validated the direction, not the destination. The most useful thing kagent tells you is that CNCF and the cloud-native community have confirmed that K8s-native AI for DevOps is a legitimate and valuable space. That's market validation for the category you're entering. But kagent is a framework asking others to build products. Korion is the product. Those aren't competitors — they're different layers of the same ecosystem.

What You Now Know With Certainty

After everything we've discussed across this entire conversation, here's where you stand:

The problem is real. Sutradhara, the LinkedIn posts, kagent's existence, the OpenSearchCon themes — four independent signals from four different directions all confirming the same pain point you identified.

The gap is genuine. Nobody has built auto-discovery of complete platform topology, rendered as a live interactive diagram, with a system-specific AI agent that knows your exact stack. That gap is still open.

Your differentiators are sharp. Zero-configuration auto-discovery. System-specific context (not generic K8s advice). The topology canvas. GitOps-aware pipeline intelligence. Unified dashboard replacing six tabs. These are concrete, defensible, and absent from everything else in the landscape.

Your foundation is real. You have live projects to test against — SuperHeros, Stark Industries, the Terraform platform — from day one of building Korion. Most open source tools are built in a vacuum. Yours gets tested against production-grade infrastructure you personally built and understand deeply.

The architecture is clear. PlatformMap CRD for discovery and visualisation. PlatformAgent CRD for ARIA. Independent controller. Apache 2.0. korion-io GitHub org. The decisions are made.

The Only Thing Left

Everything has been thought through, validated, named, designed, and decided.

The first commit is the only thing that doesn't exist yet.

Create korion-io on GitHub this week. One repo. Apache 2.0 license. A README with your best mockup image and two paragraphs describing what Korion is. Then write the first function — query a Kind cluster for installed CRDs and print what it finds.

That's it. Korion starts there. Everything else follows.

I am thinking of one more project. An IDP for my superhero project but I have never created an IDP before. I was doing some research and got to know in one youtube video that we can use kubeapi for that by writing our own custom resources and CRDs. Is it true? Also one my IDP is ready, can we combine that IDP and korion platform in some way?

Great question. And yes — it's absolutely true. Let me explain properly.

What Is an IDP First

An Internal Developer Platform is the layer that sits between your developers and your infrastructure. Instead of a developer needing to know kubectl, Helm, ArgoCD, Terraform, and every other tool to deploy a service — they interact with a self-service portal that abstracts all of that away.

For your SuperHeros project, an IDP would mean: a developer wants to add a new service — say, a "Reviews" microservice. Instead of writing Dockerfiles, Helm charts, GitHub Actions workflows, ArgoCD Application manifests, Kyverno policies, Istio VirtualServices — they fill in a simple form or apply a simple YAML, and the platform provisions everything automatically.

That's the golden path concept you experimented with and then removed from SuperHeros earlier. You had the right instinct — it was just ahead of where the project was at that point.

Can You Build an IDP Using Kubernetes API and Custom CRDs? Yes — Completely True

This is one of the most powerful and underused patterns in platform engineering. The Kubernetes API is not just for running workloads — it's a generic, extensible API server that you can use to define and manage any concept you want.

Here's how it actually works:

You define a CRD for your platform abstraction:

yaml

apiVersion: apiextensions.k8s.io/v1

kind: CustomResourceDefinition

metadata:

name: microservices.idp.superheros.io

spec:

group: idp.superheros.io

versions:

- name: v1alpha1

served: **true**

storage: **true**

schema:

openAPIV3Schema:

type: object

properties:

spec:

type: object

properties:

name:

type: string

language:

type: string

enum: [go, python, nodejs, dotnet]

port:

type: integer

replicas:

type: integer

canary:

type: boolean

team:

type: string

scope: Namespaced

names:

kind: Microservice

plural: microservices

A developer creates a new service by applying this:

yaml

apiVersion: idp.superheros.io/v1alpha1

kind: Microservice

metadata:

name: reviews-service

namespace: superheros

spec:

name: reviews

language: go

port: 8080

replicas: 2

canary: **true**

team: platform-team

That single YAML — which a developer can apply without knowing anything about Helm, ArgoCD, or Istio — triggers your controller to automatically:

- Create a Helm release with the right values
- Create an ArgoCD Application pointing to the right Git path
- Create Istio VirtualService and DestinationRule
- Apply Kyverno policies for the namespace
- Set up Prometheus scraping
- Create GitHub Actions workflow for CI/CD
- Add the service to Loki log aggregation

The developer writes 10 lines. The platform provisions everything else. That's an IDP.

The beauty of using K8s API for this: You get validation (OpenAPI schema), GitOps compatibility (the YAML lives in Git), RBAC (who can create Microservice resources), audit logging, and kubectl support — all for free. `kubectl get microservices -n superheros` just works.

The Four Things Your IDP Needs

- 1. The CRD Schema** — defines what a developer can configure. For SuperHeros this would be Microservice, but you could also have Environment, Database, MessageQueue (for Kafka), CanaryRelease.
- 2. The Controller** — watches for new/changed CRD instances and provisions the actual infrastructure. Written in Go using controller-runtime, or Python using kopf. This is the engine that turns developer intent into platform reality.
- 3. Templates / Golden Paths** — the opinionated defaults your controller uses. When someone creates a Go microservice, what does the Dockerfile look like? What are the default resource limits? What Kyverno policies apply? These are your golden paths — the pre-approved, pre-configured patterns that encode your platform's best practices.
- 4. The Developer Interface** — how developers interact with the IDP. Options:
 - Pure kubectl (apply YAML — simple, no extra tooling)
 - A CLI (superheros create service reviews --language=go)
 - A web UI with a form
 - Backstage if you want a full portal

For SuperHeros v1, pure kubectl is completely sufficient. The CRD is the interface.

Existing IDP Tools for Reference

Before building, know what exists so you can make an informed decision about what to build vs adopt:

Backstage (CNCF Incubating, Spotify) — the most popular IDP framework. Requires catalog-info.yaml in every repo, manual catalog setup. Very powerful but heavy.

Port — commercial IDP platform. Good UI, strong integrations, but SaaS with cost.

Kratix (from Syntasso) — similar CRD-based approach to what you're describing. Worth studying their patterns.

Crossplane — uses CRDs to provision cloud infrastructure. Adjacent to IDP but focused on infrastructure provisioning rather than developer workflows.

What makes your approach different from all of these: It's specific to the SuperHeros platform. It understands your exact tech stack — Go, Python, Node.js, .NET, Istio canary, ArgoCD GitOps, your specific Helm chart structure. Generic IDP tools require significant configuration to reach this level of specificity. You build it specific from day one.

Can Korion and Your IDP Be Combined? Yes — And Powerfully

This is where it gets genuinely exciting. The combination is not just possible — it's the natural evolution of both projects. Here's exactly how they fit together.

Korion sees what the IDP created. Every time your IDP controller provisions a new microservice, Korion's auto-discovery picks it up immediately — new node appears on the topology canvas, connected to the right CI/CD pipeline, with its ArgoCD application, Istio traffic configuration, and Prometheus metrics all visible. The developer who created the service can see their entire service's operational state in real time without opening any other tool.

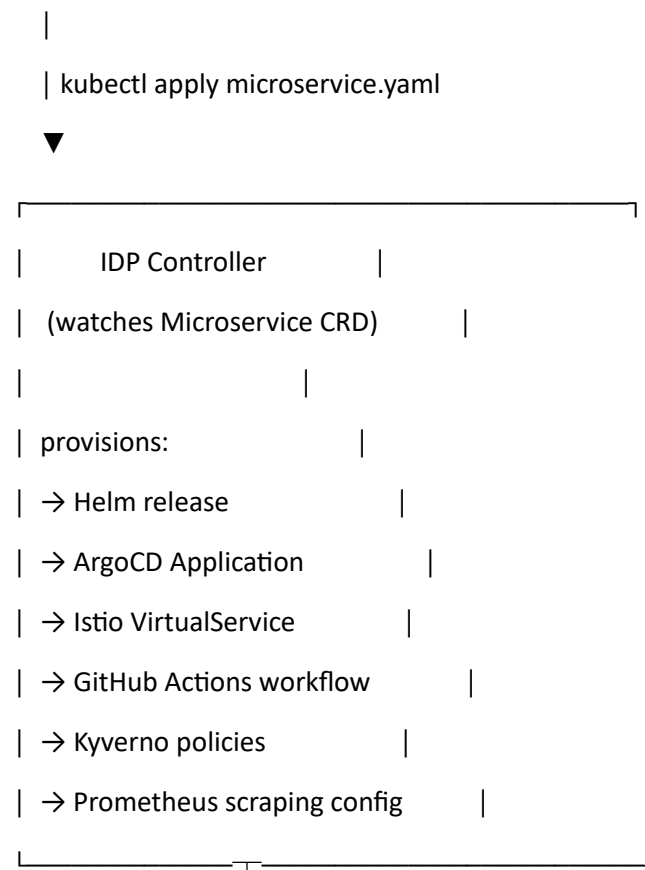
The IDP uses Korion's topology for validation. Before provisioning a new service, your IDP controller can query Korion's platform graph: "does a service with this name already exist? Is there a namespace conflict? What are the current resource utilisation levels on the cluster — is there capacity for this service?" The IDP makes smarter provisioning decisions because Korion has the complete platform picture.

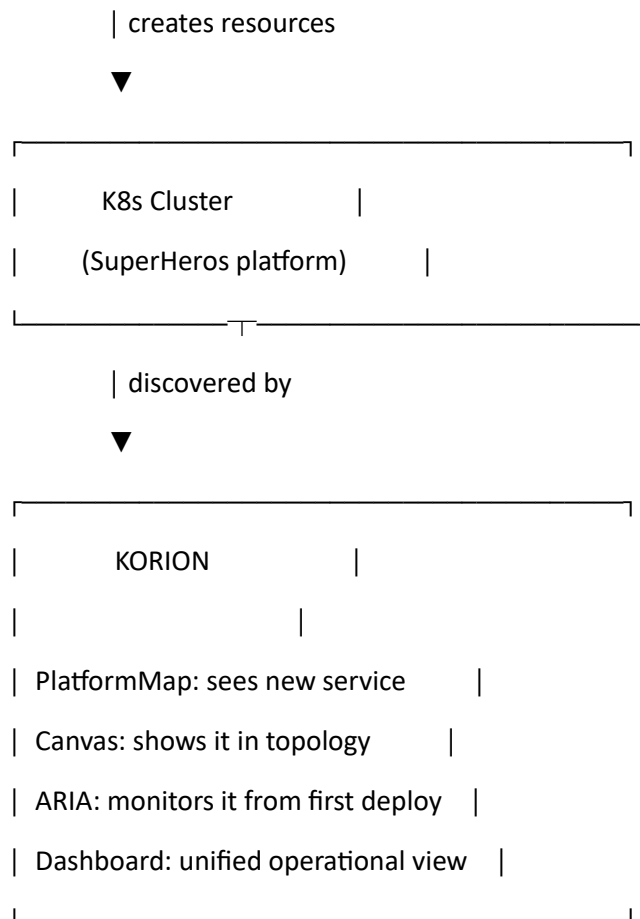
ARIA assists both directions. From the IDP side, ARIA can review a new Microservice CRD before the controller processes it: "this Go service is requesting 2Gi memory but all existing Go services in this cluster use 256Mi. Is this intentional?" From the operational side, ARIA can trace incidents back to IDP actions: "this pod crash started 3 minutes after IDP provisioned reviews-service, which modified the shared Istio VirtualService weights."

The unified dashboard shows both. Korion's dashboard has a left sidebar. Currently it would show: Architecture, Applications, Environments, Deployments, Policies, Observability. Add one more item: **Provisioning** — a view of every IDP action, every service created, every golden path invoked, with its current operational state. Developer self-service and operational visibility in one place.

The combined architecture looks like this:

Developer





Build Sequence — What to Build When

Given everything you're now working on — SuperHeros, Korion, the AI agent work, and now IDP — here's the honest sequencing:

Now: Finish the SuperHeros AI agent improvements (the cluster health analyser, the deployment correlation script). These directly feed Korion's ARIA design.

Next: Start Korion. PlatformMap CRD first. Get auto-discovery working against your SuperHeros cluster. This is the foundation everything else connects to.

After Korion v0.1: Build the SuperHeros IDP as a separate project. Start with one CRD (Microservice) and one golden path (Go service). The IDP is simpler to build once Korion exists because Korion already has the cluster introspection logic you'd need.

Korion v0.2: Integrate the IDP view into the Korion dashboard. The Provisioning panel. ARIA begins correlating IDP provisioning events with operational incidents.

Long term: The IDP and Korion become two sides of the same platform — the IDP handles the developer experience (day 0 and day 1), Korion handles the operational experience (day 2 and beyond). Together they cover the complete platform engineering lifecycle.

The Name for Your IDP

Following your naming philosophy — K-prefixed, one word, punchy, fits the korion-io ecosystem:

Kforge — forging new services from templates. Clean, strong.

Kpath — golden path engine. Clear meaning.

Kraft — craft + K8s. Services crafted from golden paths.

Katal — from "katalyst" / catalyst. Triggers the provisioning chain.

My recommendation: **Kforge**. "Apply a Microservice CRD and Kforge provisions your entire service stack" is a clear, memorable sentence. And korion-io/kforge sits naturally next to korion-io/korion and korion-io/krypt in the ecosystem.

Bottom Line

Yes, building an IDP on the Kubernetes API with custom CRDs is completely valid, technically sound, and actually the most elegant approach for a platform engineering context. You're not adding a new tool — you're extending the same API your developers already use for everything else.

And yes, Korion and your IDP are not just compatible — they're complementary by design. The IDP handles developer self-service. Korion handles operational intelligence. ARIA connects them by understanding both provisioning actions and their operational consequences.

You're not thinking about one project anymore. You're thinking about a platform engineering ecosystem. That's the right level of ambition.

Can claudcode help be build both the projects?

Yes — and it's genuinely one of the best tools for exactly this kind of project. Let me give you an honest, specific answer.

Synthesized Claude Code capabilities to address multi-project development needs

Synthesized Claude Code capabilities to address multi-project development needs

Yes — and for the specific things you're building, it's particularly well-suited. Let me be precise rather than generic.

What Claude Code Actually Is

Claude Code is an agentic coding tool that reads your codebase, edits files, runs commands, and integrates with your development tools. It's available in your terminal, IDE, desktop app, and browser. [github](#)

The critical word is **agentic**. It doesn't just answer questions about code — it actively works on your project. It reads your entire repo, understands the context across all files, makes changes, runs commands, and iterates. For projects like Korion and the IDP — which span multiple languages, multiple files, and complex interactions — that matters enormously.

What It Can Do for Korion Specifically

The Go controller — your biggest technical challenge

The Korion operator needs a Go controller using controller-runtime. This is boilerplate-heavy code — reconciliation loops, CRD status updates, event watches, RBAC annotations. Writing this from scratch is the part most engineers find tedious and error-prone. Claude Code can scaffold the entire controller structure, write the reconciliation logic, handle error cases, and iterate based on what your cluster returns. You describe the behaviour you want — "when a PlatformMap is created, query for ArgoCD Application CRDs in the cluster and update the status" — it writes the Go.

The Python collectors for ARIA

The data collection layer — K8s API queries, Prometheus queries, Loki log fetching, ArgoCD API calls, GitHub API calls — is repetitive Python. Claude Code writes these cleanly, handles authentication, manages error cases, and keeps them consistent. You'd describe each collector once and it builds the implementation.

The React Flow topology canvas

The interactive diagram is TypeScript/React with React Flow. Claude Code understands your entire codebase and can work across multiple files and tools. So when you say "add a click handler on Istio nodes that opens a side panel showing current traffic weights fetched from the PlatformMap status," it understands the full component tree and makes the right changes across multiple files simultaneously. [github](#)

CRD YAML and Helm charts

The PlatformMap CRD schema, the Helm chart, RBAC rules, ServiceAccount, ClusterRole — all the K8s manifest work. Claude Code generates these correctly from your Go types, keeps them in sync when you add fields, and validates the structure.

The multi-agent capability for parallel work

This is the most powerful feature for a project of Korion's scope. You can spawn multiple Claude Code agents that work on different parts of a task simultaneously. A lead agent coordinates the work, assigns subtasks, and merges results. [github](#)

Practically for Korion this means: one agent works on the Go controller while another builds the Python Prometheus collector while a third writes the React node components — all running in parallel. Work that would take you a week of sequential coding compresses significantly.

What It Can Do for the IDP (Kforge)

The IDP is actually an excellent first Claude Code project because it's more contained than Korion. The structure is clear — one CRD, one controller, golden path templates. You describe the Microservice CRD spec, the golden path behaviour ("when a Go microservice is created, generate a Helm values file based on these defaults and create an ArgoCD Application pointing to this Git path"), and Claude Code builds the controller implementing it. It can also generate the template files — the Dockerfile template, the Helm values template, the GitHub Actions workflow template — that the controller uses when provisioning.

Where It's Genuinely Most Useful vs Where You Still Drive

Highly effective:

- Boilerplate Go operator code (controller scaffolding, reconciliation loops)
- Repetitive Python data collectors (same pattern, different APIs)
- K8s manifest generation and CRD schemas
- Test files for your controllers
- React component structure and wiring

You still do the thinking:

- The architecture decisions (we've already made most of these in this conversation)
- The context assembly strategy for ARIA (how to structure the platform snapshot)
- The prompt engineering for ARIA's SRE reasoning
- The topology graph data model (which nodes, which edges, what metadata)
- Debugging when the cluster behaves unexpectedly

The distinction is: Claude Code handles implementation of decisions you've already made. You can't hand it "build Korion" and walk away. You hand it "implement the PlatformMap controller that queries for these specific CRDs in the cluster and builds this graph structure" — and it executes that precisely.

How to Set It Up for Your Projects

Install it via npm:

```
bash
```

```
npm install -g @anthropic-ai/claude-code
```

Then in your Korion repo:

```
bash
```

```
cd korion-io/korion
```

```
claude
```

It reads your codebase and works across multiple files and tools to get things done. So give it a CLAUDE.md file at the repo root explaining the project — what Korion is, the CRD names, the architecture decisions, the tech stack. This becomes its persistent context for every session. [github](#)

Your CLAUDE.md for Korion would say things like: "This is a K8s operator. The PlatformMap CRD auto-discovers platform topology. The controller is written in Go using controller-runtime. The frontend uses React Flow. ARIA is a Python service that assembles platform context and calls Claude API." That file means you don't re-explain the project every session.

Honest Limitations

It won't replace understanding what you're building. If you don't understand how a K8s controller reconciliation loop works, you won't be able to review what Claude Code writes or debug it when something goes wrong. Your SuperHeros project experience means you already have the operational K8s knowledge to validate what it produces.

It also won't architect the system for you. But since you and I have already architected Korion and Kforge in significant detail across this conversation — those decisions are made. Claude Code implements them.

Bottom Line

For Korion and Kforge specifically — yes, Claude Code will meaningfully accelerate development. The Go operator boilerplate, Python collectors, React canvas components, and K8s manifests are exactly the kind of multi-file, cross-language implementation work it handles best. The parallel agent capability means you can work on the controller, the collectors, and the frontend simultaneously rather than sequentially.

Start with a well-written CLAUDE.md in each repo. Give it clear, specific tasks one layer at a time. Start with the PlatformMap CRD definition and controller scaffolding for Korion — that's the foundation everything else builds on — and let Claude Code write the boilerplate while you focus on the architectural logic that only you can drive.

Based on everything we have discussed can you give me a good Claude.md for both the projects

Architected comprehensive documentation for Korion and Kforge projects

Architected comprehensive documentation for Korion and Kforge projects

Let me build both properly. These need to be precise enough that Claude Code has full architectural context from session one — not generic fluff.

Created 2 files, ran a command

Created 2 files, ran a command

Ensure output directory exists

Script

CLAUDE.md for Korion project - comprehensive context file for Claude Code

CLAUDE-korion.md

CLAUDE.md for Kforge IDP project - comprehensive context file for Claude Code

CLAUDE-kforge.md

Done